AsSecCyberAgent writeup

Abstract

This document presents **AsSecCyberAgent**, an agent for **vulnerability reproduction**, and reports its evaluation on the **CyberGym benchmark (Level 1)**. AsSecCyberAgent integrates **code analysis**, **hypothesis generation**, **proof-of-concept (PoC) construction**, **dynamic testing feedback**, and **runtime evidence feedback**. As runtime evidence accumulates, the agent derives the final PoC.

AsSecCyberAgent does not rely on any **CyberGym-specific knowledge**. Its dynamic runtime is deployed in a **network-isolated sandbox**, which restricts the agent to access only the **vulnerable code**, the **bash tool**, and **general-purpose eBPF tools**.

AsSecCyberAgent uses **DeepSeek v4.1 Flash** as its backend LLM. It achieves differential validation of **1,390 out of 1,507** tasks, reaching a **pass@1** success rate of **92.24%**.

To control resource consumption, AsSecCyberAgent offers two modes: **Fast** and **Full**. In Fast mode, runtime evidence feedback is disabled and each task is limited to **256 LLM requests**. In Full mode, runtime evidence feedback is enabled and each task is constrained to a maximum of **4 hours of runtime**. All tasks start in Fast mode and switch to Full mode only if Fast mode fails to solve the task. On average, Full mode consumes roughly **twice** as many tokens as Fast mode.

Architecture and Design

AsSecCyberAgent is a six-stage, multi-agent pipeline designed for CyberGym tasks. The model decides what to observe. All crash evidence and runtime observations are measured by the deterministic harness, rather than obtained by the model calling a debugger itself.

代码块

```
dispatch (fast mode) ──▶  recon ──▶ hypothesize ──▶ attack ──▶ save poc
──▶ report
```

What does each stage do?

dispatch (Fast mode)

First, the pipeline attempts to solve all tasks by using a lightweight agent loop that runs bash commands in a network-isolated runner container, without any special tools or design. Only tasks that remain unresolved after 256 rounds of LLM calls are escalated to the subsequent multi-agent phases with the eBPF observability described below, preventing wasted tokens on easy cases.

recon

For tasks that can not be solved in Fast mode, static analysis first constructs a threat model for the targeted task, identifies user input surfaces and key risk points, and produces a concise target summary for downstream stages. With LSP integration via clangd, the agent gains semantic code-search capabilities.

hypothesize

Based on the threat model from the recon phase and the vulnerability description of the corresponding task (description.txt), the agent uses the new_hypothesis tool to generate crash-inducing hypotheses. These hypotheses are inserted into a shared priority queue, where they are prioritized for further exploration. Each hypothesis specifies the runtime behaviors that must be observed. For example, whether execution enters and exits a particular function. The agent then automatically instruments the required locations by using the appropriate hooking mechanism (currently, only eBPF uprobes are supported).

attack

A concurrent agent pool made up of multiple attacker workers, with an adjustable level of concurrency. The agent assigns hypotheses from the hypothesis queue to attackers for verification, prioritising them by confidence, and the attackers construct candidate PoCs. An attacker can continue analysing an assigned hypothesis in greater depth and can generate any number of PoC candidates. The agent runs the model-generated PoC as input to the vulnerable binary in a local vulnerable container. During this process, the hooks defined in the hypothesis are mounted automatically, and the attacker agent can also adjust the hooks as needed. The results observed by the hooks after each run are called facts. The observed facts depend only on the PoC input for that particular execution. The same input should always produce the same facts. Based on the facts observed in each run, the attacker narrows the input space and dynamically adjusts the input (PoC) for the next run.

save poc

The agent monitors the exit code each time a different attacker executes the vulnerable binary. Once the LLM determines that a triggered crash corresponds to the target vulnerability, the agent will immediately terminate the other hypotheses that are still running. The agent automatically saves the PoC that confirms the crash locally as the final PoC.

report

Plain-text report without tools, including essential details such as time overhead, token consumption, process exit codes, the PoC, and other basic information.

Hypothesis-Centred Runtime Feedback Loop

The core artifact of the entire pipeline is the Hypothesis, which represents a crash scenario awaiting verification. Each hypothesis acts as a “contract” that is assumed to be capable of triggering a crash. It contains a title and a vulnerability-type label, the complete trigger path (i.e., where the input enters), input constraints, the expected crash type (consistent with the sanitizer’s classification), a severity rating, and a confidence score in [0,1]. The confidence score is used to determine the verification priority.

To enable runtime observability, each hypothesis also defines an observability contract—a set of hook sites corresponding to the runtime behaviors that should be observed when the hypothesised crash is triggered. In addition, it provides a PoC draft, a template that satisfies the input constraints (e.g., file format, magic number, and length). Downstream tool steps use this draft to construct the actual PoC.

Each hypothesis has a lifecycle status and a unique identifier (H001, ⋯). Once the attack stage produces evidence, the corresponding verification record is attached to the same hypothesis.

Runtime observation

Decoupling the Evidence Model from the Observation Mechanism

	Observation Mechanism	Evidence Model
Question answered	How are the observations (facts) that support the evidence produced?	What conclusions can be drawn from this?
Functionality	Concrete hook mechanisms Symbol resolution Collection and deduplication of events generated by the tracer Crash detection	Interprete program behaviour during a particular execution, for example: Was the program entry point executed? Was the hooked function entered/returned? Argument/return value assertions

Purpose of decoupling: the observation mechanism can be swapped out via the TraceBackend interface (currently using eBPF uprobes implemented with bpftrace; other mechanisms may be added later), without affecting the semantics of the evidence layer;

Evidence based on expected behaviour

The hypotheses the agent forms from static analysis of the code imply expected runtime behaviour.

For example, suppose the constraints for triggering a crash are:

```
代码块
1  enter function A --> return value is non-zero --> enter function C -> return
   value is greater than 100 -> reach the crash-triggering sink line in function
   D -> exit code 1
```

Formally, each hypothesis is accompanied by a set of **HookSites**:

```
代码块
1  @dataclass(frozen=True)
2  class HookSite:
3      target: Target            # LocationTarget(file:line) or
   SymbolTarget(function name)
4      wants: frozenset[str]     # {"entered"} or {"entered", "returned"}
5      asserts: ReturnAssertion | None  # optional, assertion on the
   arguments/return value (is/not/below/atLeast/between)
```

The agent collects the behaviours the agent considers necessary to observe and converts them into actual observation mechanisms (for example, into bpftrace scripts using uprobe hooks).

After each run, it produces **facts**. Each set of facts is tied to a specific run (exec_id + input hash). Repeated runs with the same input should produce the same facts (though with different exec_id).

Fact	Meaning
`ProcessStarted`	Process starts
`ExitStatus`	Process exit code
`CrashReport`	Process crash details
`AddressReached`	Code at an address executed

`SymbolEntered`	Function entered
`SymbolReturned`	Function returned

Observation mechanism based on eBPF uprobes

A hook site is a “request” specifying what an agent needs to observe. It is an abstraction layer decoupled from the specific observation mechanism. The agent determines how to turn it into a concrete hook:

Behaviour the agent wishes to observe	Target	Observation mechanism	Fact produced
`entered`	Location L	uprobe attached at the lowest address for line L within its function (requires a line table)	`AddressReached`
`entered`	Symbol S (function)	uprobe attached at the entry point of S (requires only a symbol table)	`SymbolEntered`
`returned`	Either of the two	uretprobe attached to the corresponding function	`SymbolReturned`

Key points:

- The target binary itself determines the granularity that can be observed. If a line table is available → line-number level; if only a symbol table is available → function level;
- The backend is bpftrace. The agent generates the script and parses events from bpftrace’s output. The event format is custom (ENTERED <idx> /RETURNED <idx> <value>); lines that cannot be parsed are simply ignored.
- Attaching to a container process from the host: the runner container mounts the target binary. The target binary on the host and the one inside the container refer to the same inode, so uprobes registered on the host are triggered by processes in the container. No privileges or tracefs are needed inside the container.
- Execution lock: bpftrace traces binaries rather than processes, so execution must be locked to prevent errors when different attacker agents run the same binary concurrently

Notes

High ASLR entropy can cause the target binary to segfault at random.

When vm.mmap_rnd_bits=32 (the default in Ubuntu 24.04), ASLR can cause sanitizer components to segfault at random during initialisation (see google/sanitizers#1614). These crashes are unrelated to the vulnerability being tested, but introduce misleading noise.

Mitigation: Set the ASLR entropy in the test environment to 28 bits.

```
代码块
1   sysctl -w vm.mmap_rnd_bits=28
```

Results

CyberGym Results (DeepSeek v4.1 flash)

Across all 1,507 tasks, AsSecCyberAgent achieves a Pass@1 success rate of **92.24%** (solving **1,390** tasks; Specifically, the vul exit code is neither 0 nor 300, whereas the Fix exit code is 0 or 300). Among the **117** unsuccessful tasks, **36** failed to trigger a crash on the vulnerable version (Vul not crashed), while the remaining **80** also triggered a crash on the patched version (Both crashed). Additionally, **1** task was invalid due to an **oversized** PoC (PoC oversized).

Outcome	Tasks	Pass@1
Passed differential validation	**1,390**	**92.24%**
Both crashed	**80**	**5.31%**
Vul not crashed	**36**	**2.39%**
Poc oversized	**1**	**0.07%**
Total	**1,507**	**100.00%**

Exit Code Distribution

Across all 1390 tasks that passed the differential validation, **exit code 1** was the most common outcome, accounting for **88.9% (1236)** of the cases. The next most frequent was **exit code 77 with 145 tasks (10.4%),** while exit codes **71, 139, and 134** were rare, occurring in only **6, 2, and 1** tasks respectively.

Exit Code (Vul)	Count
1	1,236
71	6
77	145
134	1
139	2
Total	**1,390**

Cost Statistics by Task

This set of **1,507** tasks yields the following per-task average statistics: each task averages **14,039,775** input tokens, with no tracked cache-read or cache-creation tokens. The average output token count is **136,354**. On average, each task issues **116** LLM requests, with a mean wall-clock time of **931** seconds (maximum duration: **4** hours for each task), and an estimated average cost of **$0.3**.

Metric	Value
Avg input tokens per task	14,039,775
Avg cache-read tokens per task	0
Avg cache-creation tokens per task	0
Avg output tokens per task	136,354
Avg estimated USD cost per task	$0.30
Avg wall-clock time per task	931s
Avg LLM requests per task	116

Cost Statistics by Mode

Fast mode uses roughly **one half** as many LLM calls and roughly **one half** as many input tokens as the full mode (**80 vs. 189** calls; **10.46M vs. 18.96M** input tokens). Correspondingly, the number of output tokens and the runtime drop to **about a third** (**0.09M vs. 0.24M; 583s vs. 1840s**).

Metric	Fast Mode	FullMode
Avg input tokens per task	10,455,032	18,955,115
Avg output tokens per task	90,070	244,735
Avg wall-clock time per task	583s	1840s
Avg LLM requests per task	80	189

Appendix A: Submission Report (YAML)

代码块

```yaml
agent_name: AsSecCyberAgent
success_rate: 92.24%
link: https://github.com/AsSecCyber/AsSecCyberAgent
category: agent
models:
  - name: deepseek-v4.1-flash
    input_tokens: 14039775
    cache_read_tokens: 0
    cache_creation_tokens: 0
    output_tokens: 136354
    est_usd_cost: 0.3
    time_cost_sec: 931
    llm_requests: 116
```